

10 | Attention Mechanisms

The optic nerve of a primate's visual system receives massive sensory input, far exceeding what the brain can fully process. Fortunately, not all stimuli are created equal. Focalization and concentration of consciousness have enabled primates to direct attention to objects of interest, such as preys and predators, in the complex visual environment. The ability of paying attention to only a small fraction of the information has evolutionary significance, allowing human beings to live and succeed.

Scientists have been studying attention in the cognitive neuroscience field since the 19th century. In this chapter, we will begin by reviewing a popular framework explaining how attention is deployed in a visual scene. Inspired by the attention cues in this framework, we will design models that leverage such attention cues. Notably, the Nadaraya-Watson kernel regression in 1964 is a simple demonstration of machine learning with *attention mechanisms*.

Next, we will go on to introduce attention functions that have been extensively used in the design of attention models in deep learning. Specifically, we will show how to use these functions to design the *Bahdanau attention*, a groundbreaking attention model in deep learning that can align bidirectionally and is differentiable.

In the end, equipped with the more recent *multi-head attention* and *self-attention* designs, we will describe the *transformer* architecture based solely on attention mechanisms. Since their proposal in 2017, transformers have been pervasive in modern deep learning applications, such as in areas of language, vision, speech, and reinforcement learning.

10.1 Attention Cues

Thank you for your attention to this book. Attention is a scarce resource: at the moment you are reading this book and ignoring the rest. Thus, similar to money, your attention is being paid with an opportunity cost. To ensure that your investment of attention right now is worthwhile, we have been highly motivated to pay our attention carefully to produce a nice book. Attention is the keystone in the arch of life and holds the key to any work's exceptionalism.

Since economics studies the allocation of scarce resources, we are in the era of the attention economy, where human attention is treated as a limited, valuable, and scarce commodity that can be exchanged. Numerous business models have been developed to capitalize on it. On music or video streaming services, we either pay attention to their ads or pay money to hide them. For growth in the world of online games, we either pay attention to participate in battles, which attract new gamers, or pay money to instantly become powerful. Nothing comes for free.

All in all, information in our environment is not scarce, attention is. When inspecting a visual scene, our optic nerve receives information at the order of 10^8 bits per second, far exceeding what our brain can fully process. Fortunately, our ancestors had learned from experience (also known

as data) that *not all sensory inputs are created equal*. Throughout human history, the capability of directing attention to only a fraction of information of interest has enabled our brain to allocate resources more smartly to survive, to grow, and to socialize, such as detecting predators, preys, and mates.

10.1.1 Attention Cues in Biology

To explain how our attention is deployed in the visual world, a two-component framework has emerged and been pervasive. This idea dates back to William James in the 1890s, who is considered the “father of American psychology” (James, 2007). In this framework, subjects selectively direct the spotlight of attention using both the *nonvolitional cue* and *volitional cue*.

The nonvolitional cue is based on the saliency and conspicuity of objects in the environment. Imagine there are five objects in front of you: a newspaper, a research paper, a cup of coffee, a notebook, and a book such as in Fig. 10.1.1. While all the paper products are printed in black and white, the coffee cup is red. In other words, this coffee is intrinsically salient and conspicuous in this visual environment, automatically and involuntarily drawing attention. So you bring the fovea (the center of the macula where visual acuity is highest) onto the coffee as shown in Fig. 10.1.1.

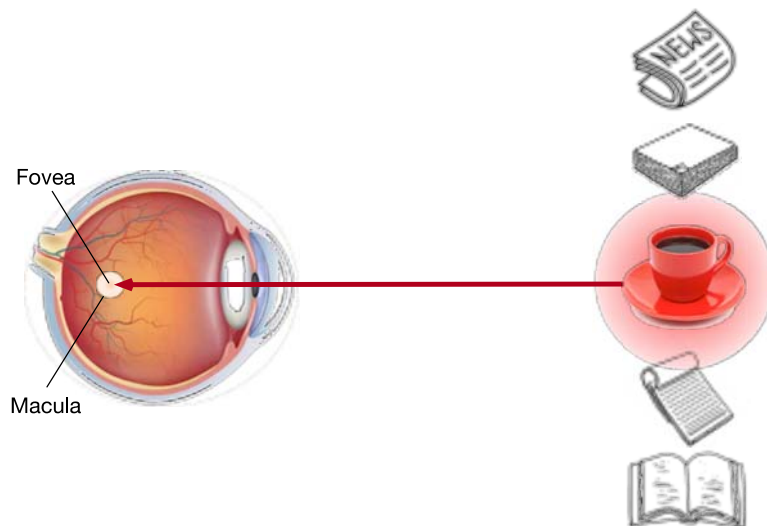


Fig. 10.1.1: Using the nonvolitional cue based on saliency (red cup, non-paper), attention is involuntarily directed to the coffee.

After drinking coffee, you become caffeinated and want to read a book. So you turn your head, refocus your eyes, and look at the book as depicted in Fig. 10.1.2. Different from the case in Fig. 10.1.1 where the coffee biases you towards selecting based on saliency, in this task-dependent case you select the book under cognitive and volitional control. Using the volitional cue based on variable selection criteria, this form of attention is more deliberate. It is also more powerful with the subject's voluntary effort.

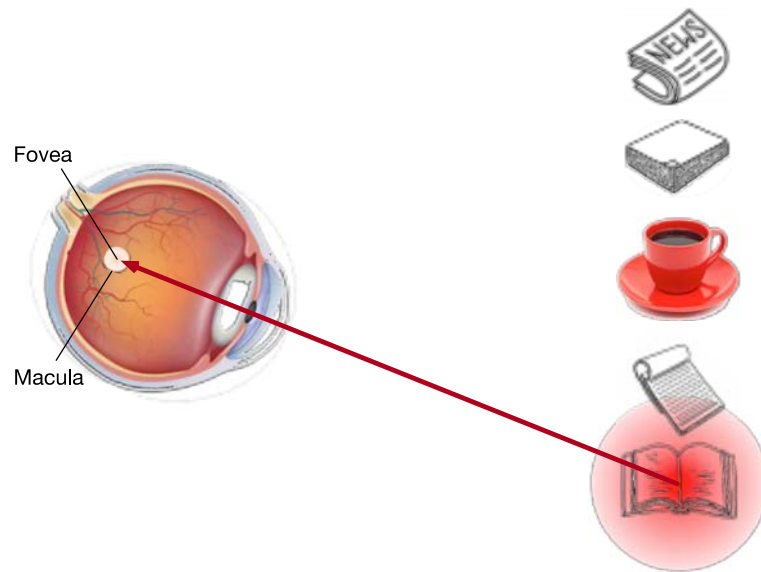


Fig. 10.1.2: Using the volitional cue (want to read a book) that is task-dependent, attention is directed to the book under volitional control.

10.1.2 Queries, Keys, and Values

Inspired by the nonvolitional and volitional attention cues that explain the attentional deployment, in the following we will describe a framework for designing attention mechanisms by incorporating these two attention cues.

To begin with, consider the simpler case where only nonvolitional cues are available. To bias selection over sensory inputs, we can simply use a parameterized fully-connected layer or even non-parameterized max or average pooling.

Therefore, what sets attention mechanisms apart from those fully-connected layers or pooling layers is the inclusion of the volitional cues. In the context of attention mechanisms, we refer to volitional cues as *queries*. Given any query, attention mechanisms bias selection over sensory inputs (e.g., intermediate feature representations) via *attention pooling*. These sensory inputs are called *values* in the context of attention mechanisms. More generally, every value is paired with a *key*, which can be thought of the nonvolitional cue of that sensory input. As shown in Fig. 10.1.3, we can design attention pooling so that the given query (volitional cue) can interact with keys (nonvolitional cues), which guides bias selection over values (sensory inputs).

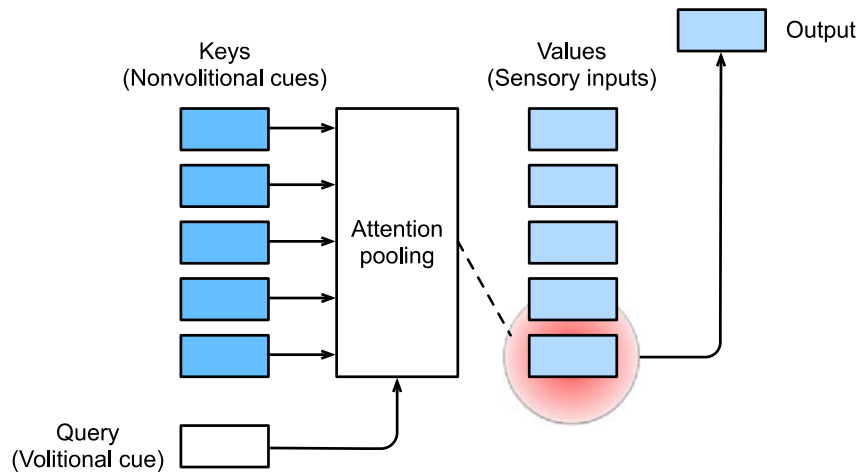


Fig. 10.1.3: Attention mechanisms bias selection over values (sensory inputs) via attention pooling, which incorporates queries (volitional cues) and keys (nonvolitional cues).

Note that there are many alternatives for the design of attention mechanisms. For instance, we can design a non-differentiable attention model that can be trained using reinforcement learning methods (Mnih et al., 2014). Given the dominance of the framework in Fig. 10.1.3, models under this framework will be the center of our attention in this chapter.

10.1.3 Visualization of Attention

Average pooling can be treated as a weighted average of inputs, where weights are uniform. In practice, attention pooling aggregates values using weighted average, where weights are computed between the given query and different keys.

```
import torch
from d2l import torch as d2l
```

To visualize attention weights, we define the `show_heatmaps` function. Its input matrices has the shape (number of rows for display, number of columns for display, number of queries, number of keys).

```
#@save
def show_heatmaps(matrices, xlabel, ylabel, titles=None, figsize=(2.5, 2.5),
                  cmap='Reds'):
    """Show heatmaps of matrices."""
    d2l.use_svg_display()
    num_rows, num_cols = matrices.shape[0], matrices.shape[1]
    fig, axes = d2l.plt.subplots(num_rows, num_cols, figsize=figsize,
                                sharex=True, sharey=True, squeeze=False)
    for i, (row_axes, row_matrices) in enumerate(zip(axes, matrices)):
        for j, (ax, matrix) in enumerate(zip(row_axes, row_matrices)):
            pcm = ax.imshow(matrix.detach().numpy(), cmap=cmap)
            if i == num_rows - 1:
                ax.set_xlabel(xlabel)
            if j == 0:
                ax.set_ylabel(ylabel)
```

(continues on next page)

```

if titles:
    ax.set_title(titles[j])
fig.colorbar(pcm, ax=axes, shrink=0.6);

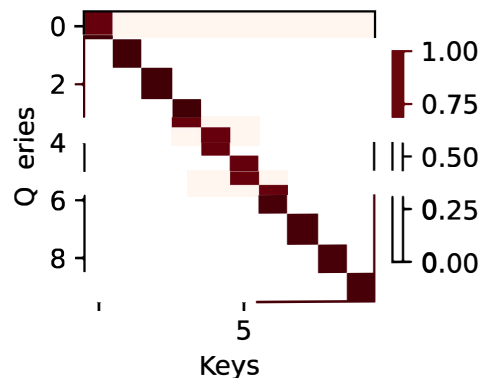
```

For demonstration, we consider a simple case where the attention weight is one only when the query and the key are the same; otherwise it is zero.

```

attention_weights = torch.eye(10).reshape((1, 1, 10, 10))
show_heatmaps(attention_weights, xlabel='Keys', ylabel='Queries')

```



In the subsequent sections, we will often invoke this function to visualize attention weights.

Summary

- Human attention is a limited, valuable, and scarce resource.
- Subjects selectively direct attention using both the nonvolitional and volitional cues. The former is based on saliency and the latter is task-dependent.
- Attention mechanisms are different from fully-connected layers or pooling layers due to inclusion of the volitional cues.
- Attention mechanisms bias selection over values (sensory inputs) via attention pooling, which incorporates queries (volitional cues) and keys (nonvolitional cues). Keys and values are paired.
- We can visualize attention weights between queries and keys.

Exercises

1. What can be the volitional cue when decoding a sequence token by token in machine translation? What are the nonvolitional cues and the sensory inputs?
2. Randomly generate a 10×10 matrix and use the softmax operation to ensure each row is a valid probability distribution. Visualize the output attention weights.

10.2 Attention Pooling: Nadaraya-Watson Kernel Regression

Now you know the major components of attention mechanisms under the framework in Fig. 10.1.3. To recapitulate, the interactions between queries (volitional cues) and keys (nonvolitional cues) result in *attention pooling*. The attention pooling selectively aggregates values (sensory inputs) to produce the output. In this section, we will describe attention pooling in greater detail to give you a high-level view of how attention mechanisms work in practice. Specifically, the Nadaraya-Watson kernel regression model proposed in 1964 is a simple yet complete example for demonstrating machine learning with attention mechanisms.

```
import torch
from torch import nn
from d2l import torch as d2l
```

10.2.1 Generating the Dataset

To keep things simple, let us consider the following regression problem: given a dataset of input-output pairs $\{(x_1, y_1), \dots, (x_n, y_n)\}$, how to learn f to predict the output $\hat{y} = f(x)$ for any new input x ?

Here we generate an artificial dataset according to the following nonlinear function with the noise term ϵ :

$$y_i = 2 \sin(x_i) + x_i^{0.8} + \epsilon, \quad (10.2.1)$$

where ϵ obeys a normal distribution with zero mean and standard deviation 0.5. Both 50 training examples and 50 testing examples are generated. To better visualize the pattern of attention later, the training inputs are sorted.

```
n_train = 50 # No. of training examples
x_train, _ = torch.sort(torch.rand(n_train) * 5) # Training inputs

def f(x):
    return 2 * torch.sin(x) + x**0.8

y_train = f(x_train) + torch.normal(0.0, 0.5, (n_train,)) # Training outputs
x_test = torch.arange(0, 5, 0.1) # Testing examples
y_truth = f(x_test) # Ground-truth outputs for the testing examples
n_test = len(x_test) # No. of testing examples
n_test
```

```
50
```

The following function plots all the training examples (represented by circles), the ground-truth data generation function f without the noise term (labeled by “Truth”), and the learned prediction function (labeled by “Pred”).

```
def plot_kernel_reg(y_hat):
    d2l.plot(x_test, [y_truth, y_hat], 'x', 'y', legend=['Truth', 'Pred'],
             xlim=[0, 5], ylim=[-1, 5])
    d2l.plt.plot(x_train, y_train, 'o', alpha=0.5);
```

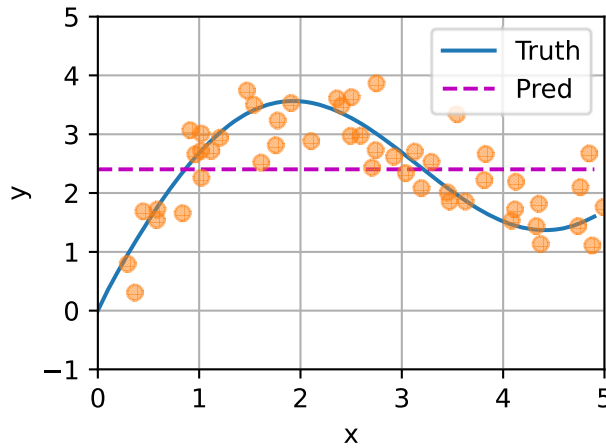
10.2.2 Average Pooling

We begin with perhaps the world’s “dumbest” estimator for this regression problem: using average pooling to average over all the training outputs:

$$f(x) = \frac{1}{n} \sum_{i=1}^n y_i, \quad (10.2.2)$$

which is plotted below. As we can see, this estimator is indeed not so smart.

```
y_hat = torch.repeat_interleave(y_train.mean(), n_test)
plot_kernel_reg(y_hat)
```



10.2.3 Nonparametric Attention Pooling

Obviously, average pooling omits the inputs x_i . A better idea was proposed by Nadaraya (Nadaraya, 1964) and Watson (Watson, 1964) to weigh the outputs y_i according to their input locations:

$$f(x) = \sum_{i=1}^n \frac{K(x - x_i)}{\sum_{j=1}^n K(x - x_j)} y_i, \quad (10.2.3)$$

where K is a *kernel*. The estimator in (10.2.3) is called *Nadaraya-Watson kernel regression*. Here we will not dive into details of kernels. Recall the framework of attention mechanisms in Fig. 10.1.3. From the perspective of attention, we can rewrite (10.2.3) in a more generalized form of *attention pooling*:

$$f(x) = \sum_{i=1}^n \alpha(x, x_i) y_i, \quad (10.2.4)$$

where x is the query and (x_i, y_i) is the key-value pair. Comparing (10.2.4) and (10.2.2), the attention pooling here is a weighted average of values y_i . The *attention weight* $\alpha(x, x_i)$ in (10.2.4) is assigned to the corresponding value y_i based on the interaction between the query x and the key x_i modeled by α . For any query, its attention weights over all the key-value pairs are a valid probability distribution: they are non-negative and sum up to one.

To gain intuitions of attention pooling, just consider a *Gaussian kernel* defined as

$$K(u) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{u^2}{2}\right). \quad (10.2.5)$$

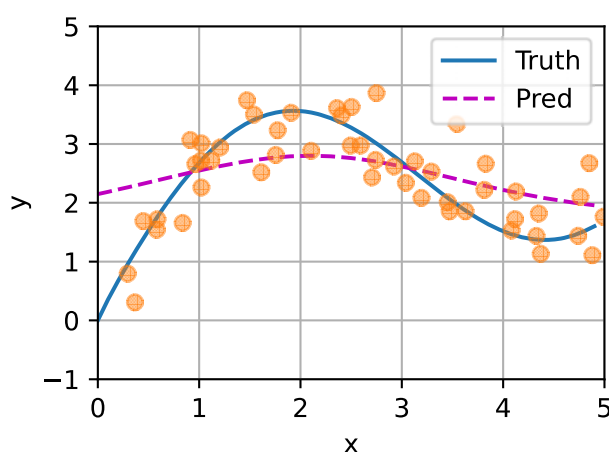
Plugging the Gaussian kernel into (10.2.4) and (10.2.3) gives

$$\begin{aligned} f(x) &= \sum_{i=1}^n \alpha(x, x_i) y_i \\ &= \sum_{i=1}^n \frac{\exp\left(-\frac{1}{2}(x - x_i)^2\right)}{\sum_{j=1}^n \exp\left(-\frac{1}{2}(x - x_j)^2\right)} y_i \\ &= \sum_{i=1}^n \text{softmax}\left(-\frac{1}{2}(x - x_i)^2\right) y_i. \end{aligned} \quad (10.2.6)$$

In (10.2.6), a key x_i that is closer to the given query x will get *more attention* via a *larger attention weight* assigned to the key's corresponding value y_i .

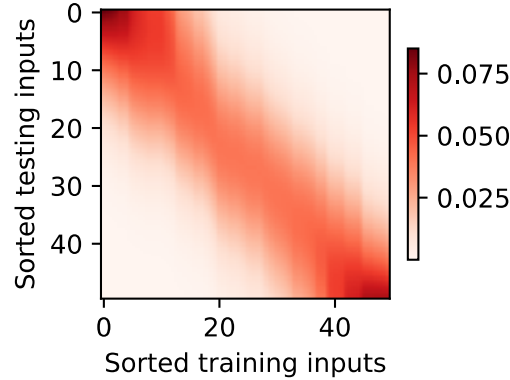
Notably, Nadaraya-Watson kernel regression is a nonparametric model; thus (10.2.6) is an example of *nonparametric attention pooling*. In the following, we plot the prediction based on this nonparametric attention model. The predicted line is smooth and closer to the ground-truth than that produced by average pooling.

```
# Shape of `X_repeat`: ('n_test', 'n_train'), where each row contains the
# same testing inputs (i.e., same queries)
X_repeat = x_test.repeat_interleave(n_train).reshape((-1, n_train))
# Note that `x_train` contains the keys. Shape of `attention_weights`:
# ('n_test', 'n_train'), where each row contains attention weights to be
# assigned among the values (`y_train`) given each query
attention_weights = nn.functional.softmax(-(X_repeat - x_train)**2 / 2, dim=1)
# Each element of `y_hat` is weighted average of values, where weights are
# attention weights
y_hat = torch.matmul(attention_weights, y_train)
plot_kernel_reg(y_hat)
```



Now let us take a look at the attention weights. Here testing inputs are queries while training inputs are keys. Since both inputs are sorted, we can see that the closer the query-key pair is, the higher attention weight is in the attention pooling.


```
d2l.show_heatmaps(attention_weights.unsqueeze(0).unsqueeze(0),
                  xlabel='Sorted training inputs',
                  ylabel='Sorted testing inputs')
```



10.2.4 Parametric Attention Pooling

Nonparametric Nadaraya-Watson kernel regression enjoys the *consistency* benefit: given enough data this model converges to the optimal solution. Nonetheless, we can easily integrate learnable parameters into attention pooling.

As an example, slightly different from (10.2.6), in the following the distance between the query x and the key x_i is multiplied by a learnable parameter w :

$$\begin{aligned} f(x) &= \sum_{i=1}^n \alpha(x, x_i) y_i \\ &= \sum_{i=1}^n \frac{\exp(-\frac{1}{2}((x - x_i)w)^2)}{\sum_{j=1}^n \exp(-\frac{1}{2}((x - x_j)w)^2)} y_i \\ &= \sum_{i=1}^n \text{softmax}\left(-\frac{1}{2}((x - x_i)w)^2\right) y_i. \end{aligned} \quad (10.2.7)$$

In the rest of the section, we will train this model by learning the parameter of the attention pooling in (10.2.7).

Batch Matrix Multiplication

To more efficiently compute attention for minibatches, we can leverage batch matrix multiplication utilities provided by deep learning frameworks.

Suppose that the first minibatch contains n matrices $\mathbf{X}_1, \dots, \mathbf{X}_n$ of shape $a \times b$, and the second minibatch contains n matrices $\mathbf{Y}_1, \dots, \mathbf{Y}_n$ of shape $b \times c$. Their batch matrix multiplication results in n matrices $\mathbf{X}_1 \mathbf{Y}_1, \dots, \mathbf{X}_n \mathbf{Y}_n$ of shape $a \times c$. Therefore, given two tensors of shape (n, a, b) and (n, b, c) , the shape of their batch matrix multiplication output is (n, a, c) .

```
X = torch.ones((2, 1, 4))
Y = torch.ones((2, 4, 6))
torch.bmm(X, Y).shape
```

```
torch.Size([2, 1, 6])
```

In the context of attention mechanisms, we can use minibatch matrix multiplication to compute weighted averages of values in a minibatch.

```
weights = torch.ones((2, 10)) * 0.1
values = torch.arange(20.0).reshape((2, 10))
torch.bmm(weights.unsqueeze(1), values.unsqueeze(-1))
```

```
tensor([[[ 4.5000]],
        [[14.5000]]])
```

Defining the Model

Using minibatch matrix multiplication, below we define the parametric version of Nadaraya-Watson kernel regression based on the parametric attention pooling in (10.2.7).

```
class NWKernelRegression(nn.Module):
    def __init__(self, **kwargs):
        super().__init__(**kwargs)
        self.w = nn.Parameter(torch.rand((1,)), requires_grad=True))

    def forward(self, queries, keys, values):
        # Shape of the output 'queries' and 'attention_weights':
        # (no. of queries, no. of key-value pairs)
        queries = queries.repeat_interleave(keys.shape[1]).reshape((-1, keys.shape[1]))
        self.attention_weights = nn.functional.softmax(
            -((queries - keys) * self.w)**2 / 2, dim=1)
        # Shape of 'values': (no. of queries, no. of key-value pairs)
        return torch.bmm(self.attention_weights.unsqueeze(1),
                        values.unsqueeze(-1)).reshape(-1)
```

Training

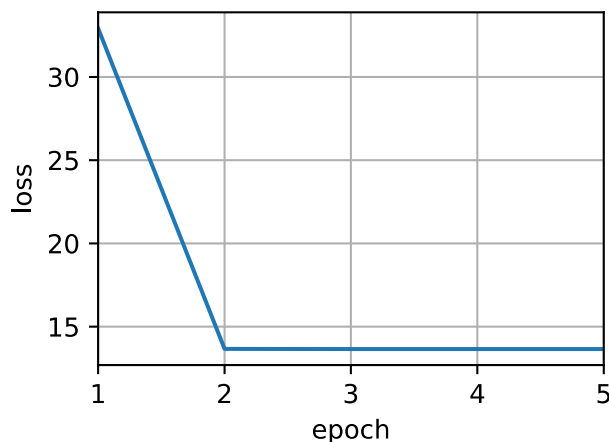
In the following, we transform the training dataset to keys and values to train the attention model. In the parametric attention pooling, any training input takes key-value pairs from all the training examples except for itself to predict its output.

```
# Shape of 'X_tile': ('n_train', 'n_train'), where each column contains the
# same training inputs
X_tile = x_train.repeat((n_train, 1))
# Shape of 'Y_tile': ('n_train', 'n_train'), where each column contains the
# same training outputs
Y_tile = y_train.repeat((n_train, 1))
# Shape of 'keys': ('n_train', 'n_train' - 1)
keys = X_tile[(1 - torch.eye(n_train)).type(torch.bool)].reshape((n_train, -1))
# Shape of 'values': ('n_train', 'n_train' - 1)
values = Y_tile[(1 - torch.eye(n_train)).type(torch.bool)].reshape((n_train, -1))
```

Using the squared loss and stochastic gradient descent, we train the parametric attention model.

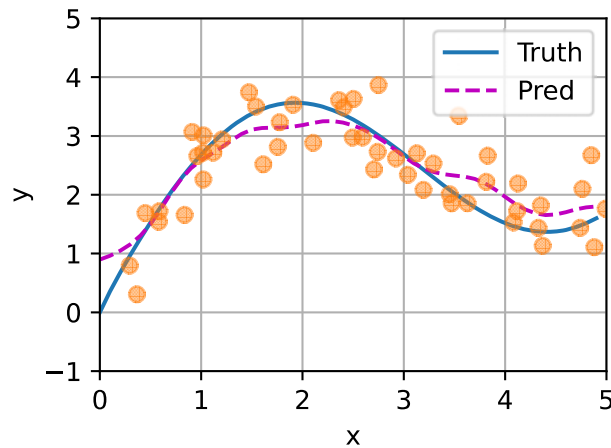
```
net = NWKernelRegression()
loss = nn.MSELoss(reduction='none')
trainer = torch.optim.SGD(net.parameters(), lr=0.5)
animator = d2l.Animator(xlabel='epoch', ylabel='loss', xlim=[1, 5])

for epoch in range(5):
    trainer.zero_grad()
    l = loss(net(x_train, keys, values), y_train)
    l.sum().backward()
    trainer.step()
    print(f'epoch {epoch + 1}, loss {float(l.sum()):.6f}')
    animator.add(epoch + 1, float(l.sum()))
```



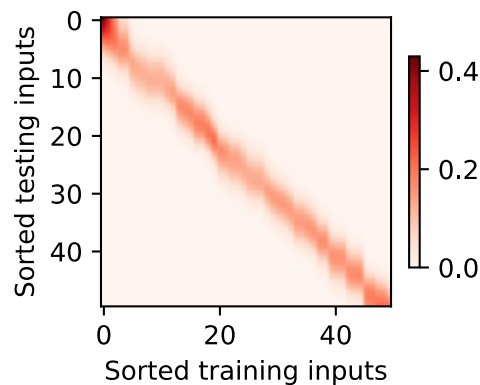
After training the parametric attention model, we can plot its prediction. Trying to fit the training dataset with noise, the predicted line is less smooth than its nonparametric counterpart that was plotted earlier.

```
# Shape of 'keys': ('n_test', 'n_train'), where each column contains the same
# training inputs (i.e., same keys)
keys = x_train.repeat((n_test, 1))
# Shape of 'value': ('n_test', 'n_train')
values = y_train.repeat((n_test, 1))
y_hat = net(x_test, keys, values).unsqueeze(1).detach()
plot_kernel_reg(y_hat)
```



Comparing with nonparametric attention pooling, the region with large attention weights becomes sharper in the learnable and parametric setting.

```
d2l.show_heatmaps(net.attention_weights.unsqueeze(0).unsqueeze(0),
                  xlabel='Sorted training inputs',
                  ylabel='Sorted testing inputs')
```



Summary

- Nadaraya-Watson kernel regression is an example of machine learning with attention mechanisms.
- The attention pooling of Nadaraya-Watson kernel regression is a weighted average of the training outputs. From the attention perspective, the attention weight is assigned to a value based on a function of a query and the key that is paired with the value.
- Attention pooling can be either nonparametric or parametric.

Exercises

1. Increase the number of training examples. Can you learn nonparametric Nadaraya-Watson kernel regression better?
2. What is the value of our learned w in the parametric attention pooling experiment? Why does it make the weighted region sharper when visualizing the attention weights?
3. How can we add hyperparameters to nonparametric Nadaraya-Watson kernel regression to predict better?
4. Design another parametric attention pooling for the kernel regression of this section. Train this new model and visualize its attention weights.

10.3 Attention Scoring Functions

In Section 10.2, we used a Gaussian kernel to model interactions between queries and keys. Treating the exponent of the Gaussian kernel in (10.2.6) as an *attention scoring function* (or *scoring function* for short), the results of this function were essentially fed into a softmax operation. As a result, we obtained a probability distribution (attention weights) over values that are paired with keys. In the end, the output of the attention pooling is simply a weighted sum of the values based on these attention weights.

At a high level, we can use the above algorithm to instantiate the framework of attention mechanisms in Fig. 10.1.3. Denoting an attention scoring function by a , Fig. 10.3.1 illustrates how the output of attention pooling can be computed as a weighted sum of values. Since attention weights are a probability distribution, the weighted sum is essentially a weighted average.

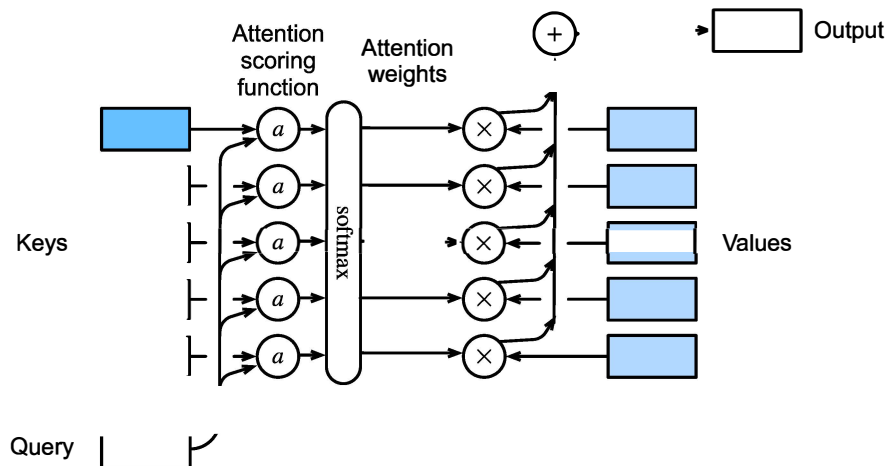


Fig. 10.3.1: Computing the output of attention pooling as a weighted average of values.

Mathematically, suppose that we have a query $\mathbf{q} \in \mathbb{R}^q$ and m key-value pairs $(\mathbf{k}_1, \mathbf{v}_1), \dots, (\mathbf{k}_m, \mathbf{v}_m)$, where any $\mathbf{k}_i \in \mathbb{R}^k$ and any $\mathbf{v}_i \in \mathbb{R}^v$. The attention pooling f is instantiated as a weighted sum of

the values:

$$f(\mathbf{q}, (\mathbf{k}_1, \mathbf{v}_1), \dots, (\mathbf{k}_m, \mathbf{v}_m)) = \sum_{i=1}^m \alpha(\mathbf{q}, \mathbf{k}_i) \mathbf{v}_i \in \mathbb{R}^v, \quad (10.3.1)$$

where the attention weight (scalar) for the query $\mathbf{q}$ and key $\mathbf{k}_i$ is computed by the softmax operation of an attention scoring function a that maps two vectors to a scalar:

$$\alpha(\mathbf{q}, \mathbf{k}_i) = \text{softmax}(a(\mathbf{q}, \mathbf{k}_i)) = \frac{\exp(a(\mathbf{q}, \mathbf{k}_i))}{\sum_{j=1}^m \exp(a(\mathbf{q}, \mathbf{k}_j))} \in \mathbb{R}. \quad (10.3.2)$$

As we can see, different choices of the attention scoring function a lead to different behaviors of attention pooling. In this section, we introduce two popular scoring functions that we will use to develop more sophisticated attention mechanisms later.

```
import math
import torch
from torch import nn
from d2l import torch as d2l
```

10.3.1 Masked Softmax Operation

As we just mentioned, a softmax operation is used to output a probability distribution as attention weights. In some cases, not all the values should be fed into attention pooling. For instance, for efficient minibatch processing in [Section 9.5](#), some text sequences are padded with special tokens that do not carry meaning. To get an attention pooling over only meaningful tokens as values, we can specify a valid sequence length (in number of tokens) to filter out those beyond this specified range when computing softmax. In this way, we can implement such a *masked softmax operation* in the following `masked_softmax` function, where any value beyond the valid length is masked as zero.

```
@save
def masked_softmax(X, valid_lens):
    """Perform softmax operation by masking elements on the last axis."""
    # `X`: 3D tensor, `valid_lens`: 1D or 2D tensor
    if valid_lens is None:
        return nn.functional.softmax(X, dim=-1)
    else:
        shape = X.shape
        if valid_lens.dim() == 1:
            valid_lens = torch.repeat_interleave(valid_lens, shape[1])
        else:
            valid_lens = valid_lens.reshape(-1)
        # On the last axis, replace masked elements with a very large negative
        # value, whose exponentiation outputs 0
        X = d2l.sequence_mask(X.reshape(-1, shape[-1]), valid_lens,
                              value=-1e6)
        return nn.functional.softmax(X.reshape(shape), dim=-1)
```

To demonstrate how this function works, consider a minibatch of two 2×4 matrix examples, where the valid lengths for these two examples are two and three, respectively. As a result of the masked softmax operation, values beyond the valid lengths are all masked as zero.

```
masked_softmax(torch.rand(2, 2, 4), torch.tensor([2, 3]))
```

```
tensor([[[[0.6025, 0.3975, 0.0000, 0.0000],
          [0.4098, 0.5902, 0.0000, 0.0000]],

        [[0.3317, 0.4265, 0.2418, 0.0000],
          [0.2789, 0.4783, 0.2428, 0.0000]]]])
```

Similarly, we can also use a two-dimensional tensor to specify valid lengths for every row in each matrix example.

```
masked_softmax(torch.rand(2, 2, 4), torch.tensor([[1, 3], [2, 4]]))
```

```
tensor([[[[1.0000, 0.0000, 0.0000, 0.0000],
          [0.3774, 0.2706, 0.3520, 0.0000]],

        [[0.5649, 0.4351, 0.0000, 0.0000],
          [0.2147, 0.2068, 0.2826, 0.2960]]]])
```

10.3.2 Additive Attention

In general, when queries and keys are vectors of different lengths, we can use additive attention as the scoring function. Given a query $\mathbf{q} \in \mathbb{R}^q$ and a key $\mathbf{k} \in \mathbb{R}^k$, the *additive attention* scoring function

$$a(\mathbf{q}, \mathbf{k}) = \mathbf{w}_v^\top \tanh(\mathbf{W}_q \mathbf{q} + \mathbf{W}_k \mathbf{k}) \in \mathbb{R}, \quad (10.3.3)$$

where learnable parameters $\mathbf{W}_q \in \mathbb{R}^{h \times q}$, $\mathbf{W}_k \in \mathbb{R}^{h \times k}$, and $\mathbf{w}_v \in \mathbb{R}^h$. Equivalent to (10.3.3), the query and the key are concatenated and fed into an MLP with a single hidden layer whose number of hidden units is h , a hyperparameter. By using $\tanh$ as the activation function and disabling bias terms, we implement additive attention in the following.

```
#@save
class AdditiveAttention(nn.Module):
    """Additive attention."""
    def __init__(self, key_size, query_size, num_hiddens, dropout, **kwargs):
        super(AdditiveAttention, self).__init__(**kwargs)
        self.W_k = nn.Linear(key_size, num_hiddens, bias=False)
        self.W_q = nn.Linear(query_size, num_hiddens, bias=False)
        self.w_v = nn.Linear(num_hiddens, 1, bias=False)
        self.dropout = nn.Dropout(dropout)

    def forward(self, queries, keys, values, valid_lens):
        queries, keys = self.W_q(queries), self.W_k(keys)
        # After dimension expansion, shape of `queries`: ('batch_size', no. of
        # queries, 1, 'num_hiddens') and shape of `keys`: ('batch_size', 1,
        # no. of key-value pairs, 'num_hiddens'). Sum them up with
        # broadcasting
        features = queries.unsqueeze(2) + keys.unsqueeze(1)
        features = torch.tanh(features)
        # There is only one output of `self.w_v`, so we remove the last
```

(continues on next page)

(continued from previous page)

```
# one-dimensional entry from the shape. Shape of `scores`:
# (`batch_size`, no. of queries, no. of key-value pairs)
scores = self.w_v(features).squeeze(-1)
self.attention_weights = masked_softmax(scores, valid_lens)
# Shape of `values`: (`batch_size`, no. of key-value pairs, value
# dimension)
return torch.bmm(self.dropout(self.attention_weights), values)
```

Let us demonstrate the above AdditiveAttention class with a toy example, where shapes (batch size, number of steps or sequence length in tokens, feature size) of queries, keys, and values are (2, 1, 20), (2, 10, 2), and (2, 10, 4), respectively. The attention pooling output has a shape of (batch size, number of steps for queries, feature size for values).

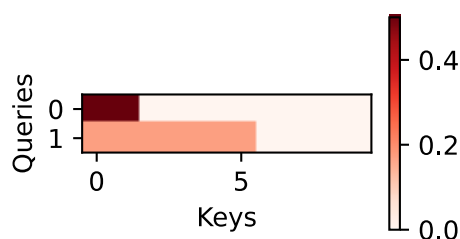
```
queries, keys = torch.normal(0, 1, (2, 1, 20)), torch.ones((2, 10, 2))
# The two value matrices in the `values` minibatch are identical
values = torch.arange(40, dtype=torch.float32).reshape(1, 10, 4).repeat(
    2, 1, 1)
valid_lens = torch.tensor([2, 6])

attention = AdditiveAttention(key_size=2, query_size=20, num_hiddens=8,
                             dropout=0.1)
attention.eval()
attention(queries, keys, values, valid_lens)
```

```
tensor([[[ 2.0000,  3.0000,  4.0000,  5.0000]],
        [[10.0000, 11.0000, 12.0000, 13.0000]]], grad_fn=<BmmBackward0>)
```

Although additive attention contains learnable parameters, since every key is the same in this example, the attention weights are uniform, determined by the specified valid lengths.

```
d2l.show_heatmaps(attention.attention_weights.reshape((1, 1, 2, 10)),
                  xlabel='Keys', ylabel='Queries')
```



10.3.3 Scaled Dot-Product Attention

A more computationally efficient design for the scoring function can be simply dot product. However, the dot product operation requires that both the query and the key have the same vector length, say d . Assume that all the elements of the query and the key are independent random variables with zero mean and unit variance. The dot product of both vectors has zero mean and a variance of d . To ensure that the variance of the dot product still remains one regardless of vector length, the *scaled dot-product attention* scoring function

$$a(\mathbf{q}, \mathbf{k}) = \mathbf{q}^\top \mathbf{k} / \sqrt{d} \quad (10.3.4)$$

divides the dot product by $\sqrt{d}$. In practice, we often think in minibatches for efficiency, such as computing attention for n queries and m key-value pairs, where queries and keys are of length d and values are of length v . The scaled dot-product attention of queries $\mathbf{Q} \in \mathbb{R}^{n \times d}$, keys $\mathbf{K} \in \mathbb{R}^{m \times d}$, and values $\mathbf{V} \in \mathbb{R}^{m \times v}$ is

$$\text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}\right) \mathbf{V} \in \mathbb{R}^{n \times v}. \quad (10.3.5)$$

In the following implementation of the scaled dot product attention, we use dropout for model regularization.

```
#@save
class DotProductAttention(nn.Module):
    """Scaled dot product attention."""
    def __init__(self, dropout, **kwargs):
        super(DotProductAttention, self).__init__(**kwargs)
        self.dropout = nn.Dropout(dropout)

    # Shape of `queries`: ('batch_size', no. of queries, 'd')
    # Shape of `keys`: ('batch_size', no. of key-value pairs, 'd')
    # Shape of `values`: ('batch_size', no. of key-value pairs, value
    # dimension)
    # Shape of `valid_lens`: ('batch_size',) or ('batch_size', no. of queries)
    def forward(self, queries, keys, values, valid_lens=None):
        d = queries.shape[-1]
        # Set `transpose_b=True` to swap the last two dimensions of `keys`
        scores = torch.bmm(queries, keys.transpose(1,2)) / math.sqrt(d)
        self.attention_weights = masked_softmax(scores, valid_lens)
        return torch.bmm(self.dropout(self.attention_weights), values)
```

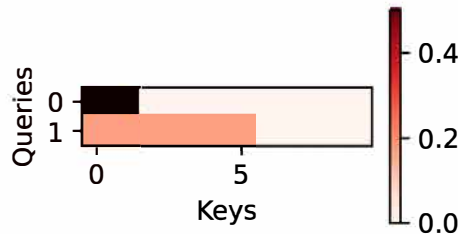
To demonstrate the above DotProductAttention class, we use the same keys, values, and valid lengths from the earlier toy example for additive attention. For the dot product operation, we make the feature size of queries the same as that of keys.

```
queries = torch.normal(0, 1, (2, 1, 2))
attention = DotProductAttention(dropout=0.5)
attention.eval()
attention(queries, keys, values, valid_lens)
```

```
tensor([[[ 2.0000,  3.0000,  4.0000,  5.0000]],
        [[10.0000, 11.0000, 12.0000, 13.0000]])
```

Same as in the additive attention demonstration, since keys contains the same element that cannot be differentiated by any query, uniform attention weights are obtained.

```
d2l.show_heatmaps(attention.attention_weights.reshape((1, 1, 2, 10)),
                  xlabel='Keys', ylabel='Queries')
```



Summary

- We can compute the output of attention pooling as a weighted average of values, where different choices of the attention scoring function lead to different behaviors of attention pooling.
- When queries and keys are vectors of different lengths, we can use the additive attention scoring function. When they are the same, the scaled dot-product attention scoring function is more computationally efficient.

Exercises

1. Modify keys in the toy example and visualize attention weights. Do additive attention and scaled dot-product attention still output the same attention weights? Why or why not?
2. Using matrix multiplications only, can you design a new scoring function for queries and keys with different vector lengths?
3. When queries and keys have the same vector length, is vector summation a better design than dot product for the scoring function? Why or why not?

10.4 Bahdanau Attention

We studied the machine translation problem in [Section 9.7](#), where we designed an encoder-decoder architecture based on two RNNs for sequence to sequence learning. Specifically, the RNN encoder transforms a variable-length sequence into a fixed-shape context variable, then the RNN decoder generates the output (target) sequence token by token based on the generated tokens and the context variable. However, even though not all the input (source) tokens are useful for decoding a certain token, the *same* context variable that encodes the entire input sequence is still used at each decoding step.

In a separate but related challenge of handwriting generation for a given text sequence, Graves designed a differentiable attention model to align text characters with the much longer pen trace, where the alignment moves only in one direction (Graves, 2013). Inspired by the idea of learning to align, Bahdanau et al. proposed a differentiable attention model without the severe unidirectional alignment limitation (Bahdanau et al., 2014). When predicting a token, if not all the input tokens are relevant, the model aligns (or attends) only to parts of the input sequence that are relevant to the current prediction. This is achieved by treating the context variable as an output of attention pooling.

10.4.1 Model

When describing Bahdanau attention for the RNN encoder-decoder below, we will follow the same notation in Section 9.7. The new attention-based model is the same as that in Section 9.7 except that the context variable $\mathbf{c}$ in (9.7.3) is replaced by $\mathbf{c}_{t'}$ at any decoding time step t' . Suppose that there are T tokens in the input sequence, the context variable at the decoding time step t' is the output of attention pooling:

$$\mathbf{c}_{t'} = \sum_{t=1}^T \alpha(\mathbf{s}_{t'-1}, \mathbf{h}_t) \mathbf{h}_t, \quad (10.4.1)$$

where the decoder hidden state $\mathbf{s}_{t'-1}$ at time step $t' - 1$ is the query, and the encoder hidden states $\mathbf{h}_t$ are both the keys and values, and the attention weight α is computed as in (10.3.2) using the additive attention scoring function defined by (10.3.3).

Slightly different from the vanilla RNN encoder-decoder architecture in Fig. 9.7.2, the same architecture with Bahdanau attention is depicted in Fig. 10.4.1.

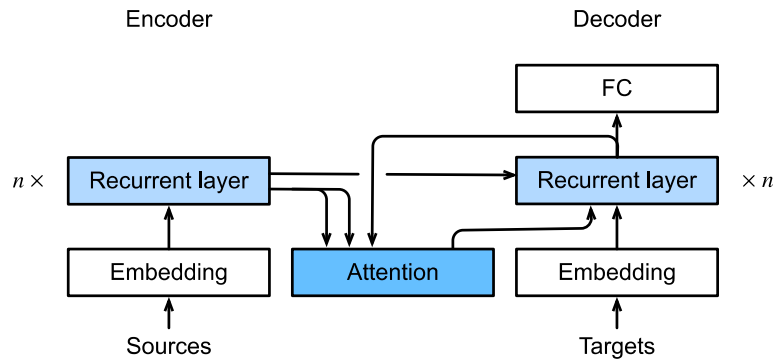


Fig. 10.4.1: Layers in an RNN encoder-decoder model with Bahdanau attention.

```
import torch
from torch import nn
from d2l import torch as d2l
```

10.4.2 Defining the Decoder with Attention

To implement the RNN encoder-decoder with Bahdanau attention, we only need to redefine the decoder. To visualize the learned attention weights more conveniently, the following Attention-Decoder class defines the base interface for decoders with attention mechanisms.

```
#@save
class AttentionDecoder(d2l.Decoder):
    """The base attention-based decoder interface."""
    def __init__(self, **kwargs):
        super(AttentionDecoder, self).__init__(**kwargs)

    @property
    def attention_weights(self):
        raise NotImplementedError
```

Now let us implement the RNN decoder with Bahdanau attention in the following Seq2SeqAttentionDecoder class. The state of the decoder is initialized with (i) the encoder final-layer hidden states at all the time steps (as keys and values of the attention); (ii) the encoder all-layer hidden state at the final time step (to initialize the hidden state of the decoder); and (iii) the encoder valid length (to exclude the padding tokens in attention pooling). At each decoding time step, the decoder final-layer hidden state at the previous time step is used as the query of the attention. As a result, both the attention output and the input embedding are concatenated as the input of the RNN decoder.

```
class Seq2SeqAttentionDecoder(AttentionDecoder):
    def __init__(self, vocab_size, embed_size, num_hiddens, num_layers,
                 dropout=0, **kwargs):
        super(Seq2SeqAttentionDecoder, self).__init__(**kwargs)
        self.attention = d2l.AdditiveAttention(
            num_hiddens, num_hiddens, num_hiddens, dropout)
        self.embedding = nn.Embedding(vocab_size, embed_size)
        self.rnn = nn.GRU(
            embed_size + num_hiddens, num_hiddens, num_layers,
            dropout=dropout)
        self.dense = nn.Linear(num_hiddens, vocab_size)

    def init_state(self, enc_outputs, enc_valid_lens, *args):
        # Shape of 'outputs': ('num_steps', 'batch_size', 'num_hiddens').
        # Shape of 'hidden_state[0]': ('num_layers', 'batch_size',
        # 'num_hiddens')
        outputs, hidden_state = enc_outputs
        return (outputs.permute(1, 0, 2), hidden_state, enc_valid_lens)

    def forward(self, X, state):
        # Shape of 'enc_outputs': ('batch_size', 'num_steps', 'num_hiddens').
        # Shape of 'hidden_state[0]': ('num_layers', 'batch_size',
        # 'num_hiddens')
        enc_outputs, hidden_state, enc_valid_lens = state
        # Shape of the output 'X': ('num_steps', 'batch_size', 'embed_size')
        X = self.embedding(X).permute(1, 0, 2)
        outputs, self._attention_weights = [], []
        for x in X:
            # Shape of 'query': ('batch_size', 1, 'num_hiddens')
```

(continues on next page)

(continued from previous page)

```
query = torch.unsqueeze(hidden_state[-1], dim=1)
# Shape of 'context': ('batch_size', 1, 'num_hiddens')
context = self.attention(
    query, enc_outputs, enc_outputs, enc_valid_lens)
# Concatenate on the feature dimension
x = torch.cat((context, torch.unsqueeze(x, dim=1)), dim=-1)
# Reshape 'x' as (1, 'batch_size', 'embed_size' + 'num_hiddens')
out, hidden_state = self.rnn(x.permute(1, 0, 2), hidden_state)
outputs.append(out)
self._attention_weights.append(self.attention.attention_weights)
# After fully-connected layer transformation, shape of 'outputs':
# ('num_steps', 'batch_size', 'vocab_size')
outputs = self.dense(torch.cat(outputs, dim=0))
return outputs.permute(1, 0, 2), [enc_outputs, hidden_state,
                                   enc_valid_lens]

@property
def attention_weights(self):
    return self._attention_weights
```

In the following, we test the implemented decoder with Bahdanau attention using a minibatch of 4 sequence inputs of 7 time steps.

```
encoder = d2l.Seq2SeqEncoder(vocab_size=10, embed_size=8, num_hiddens=16,
                             num_layers=2)
encoder.eval()
decoder = Seq2SeqAttentionDecoder(vocab_size=10, embed_size=8, num_hiddens=16,
                                  num_layers=2)
decoder.eval()
X = torch.zeros((4, 7), dtype=torch.long) # ('batch_size', 'num_steps')
state = decoder.init_state(encoder(X), None)
output, state = decoder(X, state)
output.shape, len(state), state[0].shape, len(state[1]), state[1][0].shape
```

```
(torch.Size([4, 7, 10]), 3, torch.Size([4, 7, 16]), 2, torch.Size([4, 16]))
```

10.4.3 Training

Similar to [Section 9.7.4](#), here we specify hyperparameters, instantiate an encoder and a decoder with Bahdanau attention, and train this model for machine translation. Due to the newly added attention mechanism, this training is much slower than that in [Section 9.7.4](#) without attention mechanisms.

```
embed_size, num_hiddens, num_layers, dropout = 32, 32, 2, 0.1
batch_size, num_steps = 64, 10
lr, num_epochs, device = 0.005, 250, d2l.try_gpu()

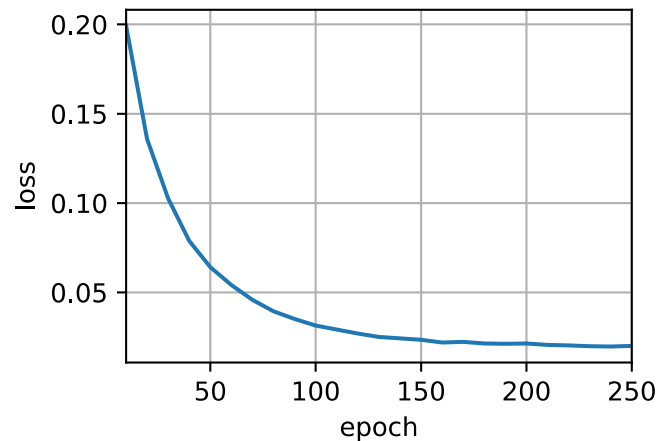
train_iter, src_vocab, tgt_vocab = d2l.load_data_nmt(batch_size, num_steps)
encoder = d2l.Seq2SeqEncoder(
    len(src_vocab), embed_size, num_hiddens, num_layers, dropout)
decoder = Seq2SeqAttentionDecoder(
```

(continues on next page)

(continued from previous page)

```
len(tgt_vocab), embed_size, num_hiddens, num_layers, dropout)
net = d2l.EncoderDecoder(encoder, decoder)
d2l.train_seq2seq(net, train_iter, lr, num_epochs, tgt_vocab, device)
```

```
loss 0.020, 5833.6 tokens/sec on cuda:0
```



After the model is trained, we use it to translate a few English sentences into French and compute their BLEU scores.

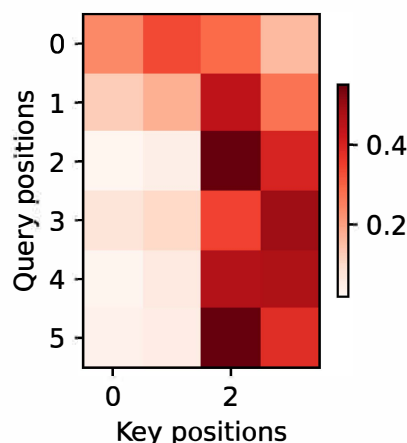
```
engs = ['go .', 'i lost .', 'he\'s calm .', 'i\'m home .']
fras = ['va !', 'j\'ai perdu .', 'il est calme .', 'je suis chez moi .']
for eng, fra in zip(engs, fras):
    translation, dec_attention_weight_seq = d2l.predict_seq2seq(
        net, eng, src_vocab, tgt_vocab, num_steps, device, True)
    print(f'{eng} => {translation}, ',
          f'bleu {d2l.bleu(translation, fra, k=2):.3f}')
```

```
go . => va !, bleu 1.000
i lost . => j'ai perdu ., bleu 1.000
he's calm . => il est riche ., bleu 0.658
i'm home . => je suis chez moi ., bleu 1.000
```

```
attention_weights = torch.cat([step[0][0][0] for step in dec_attention_weight_seq], 0).
    ↪ reshape(
        1, 1, -1, num_steps))
```

By visualizing the attention weights when translating the last English sentence, we can see that each query assigns non-uniform weights over key-value pairs. It shows that at each decoding step, different parts of the input sequences are selectively aggregated in the attention pooling.

```
# Plus one to include the end-of-sequence token
d2l.show_heatmaps(
    attention_weights[:, :, :, :len(engs[-1].split()) + 1].cpu(),
    xlabel='Key positions', ylabel='Query positions')
```



Summary

- When predicting a token, if not all the input tokens are relevant, the RNN encoder-decoder with Bahdanau attention selectively aggregates different parts of the input sequence. This is achieved by treating the context variable as an output of additive attention pooling.
- In the RNN encoder-decoder, Bahdanau attention treats the decoder hidden state at the previous time step as the query, and the encoder hidden states at all the time steps as both the keys and values.

Exercises

1. Replace GRU with LSTM in the experiment.
2. Modify the experiment to replace the additive attention scoring function with the scaled dot-product. How does it influence the training efficiency?

10.5 Multi-Head Attention

In practice, given the same set of queries, keys, and values we may want our model to combine knowledge from different behaviors of the same attention mechanism, such as capturing dependencies of various ranges (e.g., shorter-range vs. longer-range) within a sequence. Thus, it may be beneficial to allow our attention mechanism to jointly use different representation subspaces of queries, keys, and values.

To this end, instead of performing a single attention pooling, queries, keys, and values can be transformed with h independently learned linear projections. Then these h projected queries, keys, and values are fed into attention pooling in parallel. In the end, h attention pooling outputs are concatenated and transformed with another learned linear projection to produce the final output. This design is called *multi-head attention*, where each of the h attention pooling outputs is a *head* (Vaswani et al., 2017). Using fully-connected layers to perform learnable linear transformations, Fig. 10.5.1 describes multi-head attention.

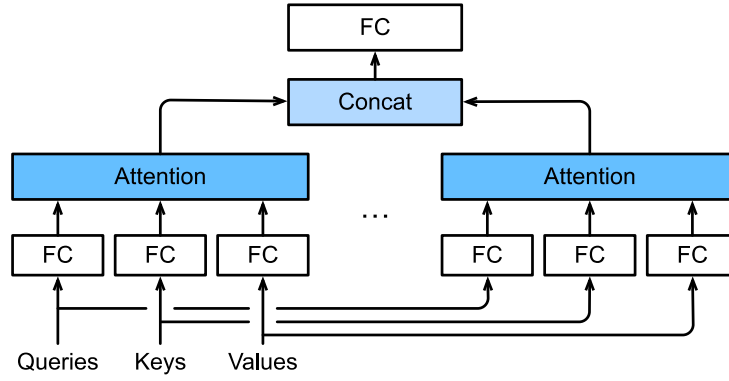


Fig. 10.5.1: Multi-head attention, where multiple heads are concatenated then linearly transformed.

10.5.1 Model

Before providing the implementation of multi-head attention, let us formalize this model mathematically. Given a query $\mathbf{q} \in \mathbb{R}^{d_q}$, a key $\mathbf{k} \in \mathbb{R}^{d_k}$, and a value $\mathbf{v} \in \mathbb{R}^{d_v}$, each attention head $\mathbf{h}_i$ ($i = 1, \dots, h$) is computed as

$$\mathbf{h}_i = f(\mathbf{W}_i^{(q)} \mathbf{q}, \mathbf{W}_i^{(k)} \mathbf{k}, \mathbf{W}_i^{(v)} \mathbf{v}) \in \mathbb{R}^{p_v}, \quad (10.5.1)$$

where learnable parameters $\mathbf{W}_i^{(q)} \in \mathbb{R}^{p_q \times d_q}$, $\mathbf{W}_i^{(k)} \in \mathbb{R}^{p_k \times d_k}$ and $\mathbf{W}_i^{(v)} \in \mathbb{R}^{p_v \times d_v}$, and f is attention pooling, such as additive attention and scaled dot-product attention in Section 10.3. The multi-head attention output is another linear transformation via learnable parameters $\mathbf{W}_o \in \mathbb{R}^{p_o \times hp_v}$ of the concatenation of h heads:

$$\mathbf{W}_o \begin{bmatrix} \mathbf{h}_1 \\ \vdots \\ \mathbf{h}_h \end{bmatrix} \in \mathbb{R}^{p_o}. \quad (10.5.2)$$

Based on this design, each head may attend to different parts of the input. More sophisticated functions than the simple weighted average can be expressed.

```
import math
import torch
from torch import nn
from d2l import torch as d2l
```

10.5.2 Implementation

In our implementation, we choose the scaled dot-product attention for each head of the multi-head attention. To avoid significant growth of computational cost and parameterization cost, we set $p_q = p_k = p_v = p_o/h$. Note that h heads can be computed in parallel if we set the number of outputs of linear transformations for the query, key, and value to $p_q h = p_k h = p_v h = p_o$. In the following implementation, p_o is specified via the argument `num_hiddens`.


```

#@save
class MultiHeadAttention(nn.Module):
    """Multi-head attention."""
    def __init__(self, key_size, query_size, value_size, num_hiddens,
                  num_heads, dropout, bias=False, **kwargs):
        super(MultiHeadAttention, self).__init__(**kwargs)
        self.num_heads = num_heads
        self.attention = d2l.DotProductAttention(dropout)
        self.W_q = nn.Linear(query_size, num_hiddens, bias=bias)
        self.W_k = nn.Linear(key_size, num_hiddens, bias=bias)
        self.W_v = nn.Linear(value_size, num_hiddens, bias=bias)
        self.W_o = nn.Linear(num_hiddens, num_hiddens, bias=bias)

    def forward(self, queries, keys, values, valid_lens):
        # Shape of 'queries', 'keys', or 'values':
        # ('batch_size', no. of queries or key-value pairs, 'num_hiddens')
        # Shape of 'valid_lens':
        # ('batch_size',) or ('batch_size', no. of queries)
        # After transposing, shape of output 'queries', 'keys', or 'values':
        # ('batch_size' * 'num_heads', no. of queries or key-value pairs,
        # 'num_hiddens' / 'num_heads')
        queries = transpose_qkv(self.W_q(queries), self.num_heads)
        keys = transpose_qkv(self.W_k(keys), self.num_heads)
        values = transpose_qkv(self.W_v(values), self.num_heads)

        if valid_lens is not None:
            # On axis 0, copy the first item (scalar or vector) for
            # 'num_heads' times, then copy the next item, and so on
            valid_lens = torch.repeat_interleave(
                valid_lens, repeats=self.num_heads, dim=0)

        # Shape of 'output': ('batch_size' * 'num_heads', no. of queries,
        # 'num_hiddens' / 'num_heads')
        output = self.attention(queries, keys, values, valid_lens)

        # Shape of 'output_concat':
        # ('batch_size', no. of queries, 'num_hiddens')
        output_concat = transpose_output(output, self.num_heads)
        return self.W_o(output_concat)

```

To allow for parallel computation of multiple heads, the above MultiHeadAttention class uses two transposition functions as defined below. Specifically, the transpose_output function reverses the operation of the transpose_qkv function.

```

#@save
def transpose_qkv(X, num_heads):
    """Transposition for parallel computation of multiple attention heads."""
    # Shape of input 'X':
    # ('batch_size', no. of queries or key-value pairs, 'num_hiddens').
    # Shape of output 'X':
    # ('batch_size', no. of queries or key-value pairs, 'num_heads',
    # 'num_hiddens' / 'num_heads')
    X = X.reshape(X.shape[0], X.shape[1], num_heads, -1)

    # Shape of output 'X':

```

(continues on next page)

(continued from previous page)

```
# ('batch_size', 'num_heads', no. of queries or key-value pairs,
# 'num_hiddens' / 'num_heads')
X = X.permute(0, 2, 1, 3)

# Shape of 'output':
# ('batch_size' * 'num_heads', no. of queries or key-value pairs,
# 'num_hiddens' / 'num_heads')
return X.reshape(-1, X.shape[2], X.shape[3])

#@save
def transpose_output(X, num_heads):
    """Reverse the operation of 'transpose_qkv'."""
    X = X.reshape(-1, num_heads, X.shape[1], X.shape[2])
    X = X.permute(0, 2, 1, 3)
    return X.reshape(X.shape[0], X.shape[1], -1)
```

Let us test our implemented MultiHeadAttention class using a toy example where keys and values are the same. As a result, the shape of the multi-head attention output is (batch_size, num_queries, num_hiddens).

```
num_hiddens, num_heads = 100, 5
attention = MultiHeadAttention(num_hiddens, num_hiddens, num_hiddens,
                              num_hiddens, num_heads, 0.5)
attention.eval()
```

```
MultiHeadAttention(
  (attention): DotProductAttention(
    (dropout): Dropout(p=0.5, inplace=False)
  )
  (W_q): Linear(in_features=100, out_features=100, bias=False)
  (W_k): Linear(in_features=100, out_features=100, bias=False)
  (W_v): Linear(in_features=100, out_features=100, bias=False)
  (W_o): Linear(in_features=100, out_features=100, bias=False)
)
```

```
batch_size, num_queries, num_kvpairs, valid_lens = 2, 4, 6, torch.tensor([3, 2])
X = torch.ones((batch_size, num_queries, num_hiddens))
Y = torch.ones((batch_size, num_kvpairs, num_hiddens))
attention(X, Y, Y, valid_lens).shape
```

```
torch.Size([2, 4, 100])
```

Summary

- Multi-head attention combines knowledge of the same attention pooling via different representation subspaces of queries, keys, and values.
- To compute multiple heads of multi-head attention in parallel, proper tensor manipulation is needed.

Exercises

1. Visualize attention weights of multiple heads in this experiment.
2. Suppose that we have a trained model based on multi-head attention and we want to prune least important attention heads to increase the prediction speed. How can we design experiments to measure the importance of an attention head?

10.6 Self-Attention and Positional Encoding

In deep learning, we often use CNNs or RNNs to encode a sequence. Now with attention mechanisms, imagine that we feed a sequence of tokens into attention pooling so that the same set of tokens act as queries, keys, and values. Specifically, each query attends to all the key-value pairs and generates one attention output. Since the queries, keys, and values come from the same place, this performs *self-attention* (Lin et al., 2017b; Vaswani et al., 2017), which is also called *intra-attention* (Cheng et al., 2016; Parikh et al., 2016; Paulus et al., 2017). In this section, we will discuss sequence encoding using self-attention, including using additional information for the sequence order.

```
import math
import torch
from torch import nn
from d2l import torch as d2l
```

10.6.1 Self-Attention

Given a sequence of input tokens $\mathbf{x}_1, \dots, \mathbf{x}_n$ where any $\mathbf{x}_i \in \mathbb{R}^d$ ($1 \leq i \leq n$), its self-attention outputs a sequence of the same length $\mathbf{y}_1, \dots, \mathbf{y}_n$, where

$$\mathbf{y}_i = f(\mathbf{x}_i, (\mathbf{x}_1, \mathbf{x}_1), \dots, (\mathbf{x}_n, \mathbf{x}_n)) \in \mathbb{R}^d \quad (10.6.1)$$

according to the definition of attention pooling f in (10.2.4). Using multi-head attention, the following code snippet computes the self-attention of a tensor with shape (batch size, number of time steps or sequence length in tokens, d). The output tensor has the same shape.

```
num_hiddens, num_heads = 100, 5
attention = d2l.MultiHeadAttention(num_hiddens, num_hiddens, num_hiddens,
                                   num_hiddens, num_heads, 0.5)
attention.eval()
```

```
MultiHeadAttention(
  (attention): DotProductAttention(
    (dropout): Dropout(p=0.5, inplace=False)
  )
  (W_q): Linear(in_features=100, out_features=100, bias=False)
  (W_k): Linear(in_features=100, out_features=100, bias=False)
  (W_v): Linear(in_features=100, out_features=100, bias=False)
  (W_o): Linear(in_features=100, out_features=100, bias=False)
)
```

```
batch_size, num_queries, valid_lens = 2, 4, torch.tensor([3, 2])
X = torch.ones((batch_size, num_queries, num_hiddens))
attention(X, X, X, valid_lens).shape
```

```
torch.Size([2, 4, 100])
```

10.6.2 Comparing CNNs, RNNs, and Self-Attention

Let us compare architectures for mapping a sequence of n tokens to another sequence of equal length, where each input or output token is represented by a d -dimensional vector. Specifically, we will consider CNNs, RNNs, and self-attention. We will compare their computational complexity, sequential operations, and maximum path lengths. Note that sequential operations prevent parallel computation, while a shorter path between any combination of sequence positions makes it easier to learn long-range dependencies within the sequence (Hochreiter et al., 2001).

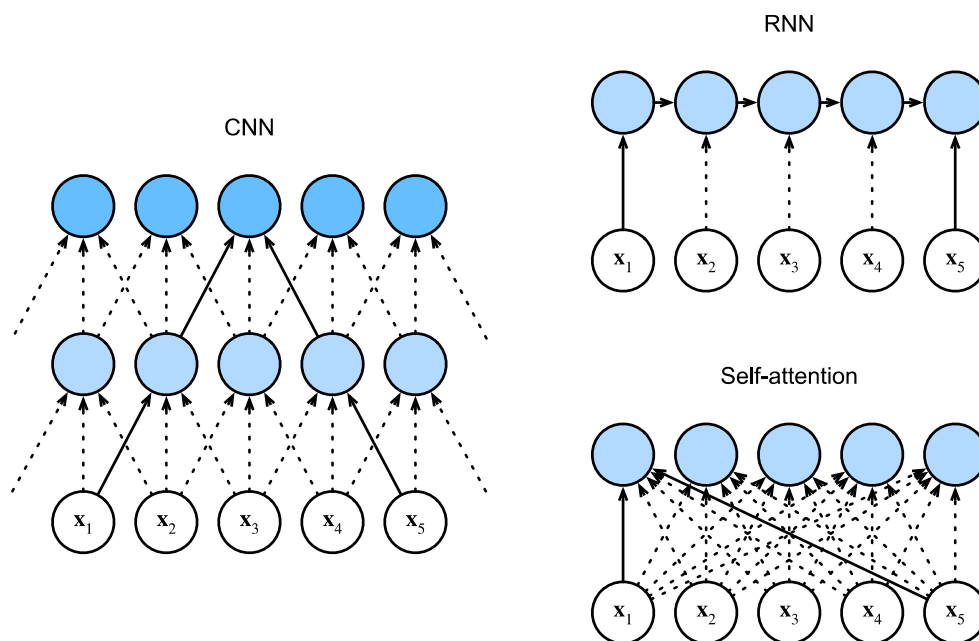


Fig. 10.6.1: Comparing CNN (padding tokens are omitted), RNN, and self-attention architectures.

Consider a convolutional layer whose kernel size is k . We will provide more details about sequence processing using CNNs in later chapters. For now, we only need to know that since the sequence

length is n , the numbers of input and output channels are both d , the computational complexity of the convolutional layer is $\mathcal{O}(knd^2)$. As Fig. 10.6.1 shows, CNNs are hierarchical so there are $\mathcal{O}(1)$ sequential operations and the maximum path length is $\mathcal{O}(n/k)$. For example, $\mathbf{x}_1$ and $\mathbf{x}_5$ are within the receptive field of a two-layer CNN with kernel size 3 in Fig. 10.6.1.

When updating the hidden state of RNNs, multiplication of the $d \times d$ weight matrix and the d -dimensional hidden state has a computational complexity of $\mathcal{O}(d^2)$. Since the sequence length is n , the computational complexity of the recurrent layer is $\mathcal{O}(nd^2)$. According to Fig. 10.6.1, there are $\mathcal{O}(n)$ sequential operations that cannot be parallelized and the maximum path length is also $\mathcal{O}(n)$.

In self-attention, the queries, keys, and values are all $n \times d$ matrices. Consider the scaled dot-product attention in (10.3.5), where a $n \times d$ matrix is multiplied by a $d \times n$ matrix, then the output $n \times n$ matrix is multiplied by a $n \times d$ matrix. As a result, the self-attention has a $\mathcal{O}(n^2d)$ computational complexity. As we can see in Fig. 10.6.1, each token is directly connected to any other token via self-attention. Therefore, computation can be parallel with $\mathcal{O}(1)$ sequential operations and the maximum path length is also $\mathcal{O}(1)$.

All in all, both CNNs and self-attention enjoy parallel computation and self-attention has the shortest maximum path length. However, the quadratic computational complexity with respect to the sequence length makes self-attention prohibitively slow for very long sequences.

10.6.3 Positional Encoding

Unlike RNNs that recurrently process tokens of a sequence one by one, self-attention ditches sequential operations in favor of parallel computation. To use the sequence order information, we can inject absolute or relative positional information by adding *positional encoding* to the input representations. Positional encodings can be either learned or fixed. In the following, we describe a fixed positional encoding based on sine and cosine functions (Vaswani et al., 2017).

Suppose that the input representation $\mathbf{X} \in \mathbb{R}^{n \times d}$ contains the d -dimensional embeddings for n tokens of a sequence. The positional encoding outputs $\mathbf{X} + \mathbf{P}$ using a positional embedding matrix $\mathbf{P} \in \mathbb{R}^{n \times d}$ of the same shape, whose element on the i^{th} row and the $(2j)^{\text{th}}$ or the $(2j + 1)^{\text{th}}$ column is

$$\begin{aligned} p_{i,2j} &= \sin\left(\frac{i}{10000^{2j/d}}\right), \\ p_{i,2j+1} &= \cos\left(\frac{i}{10000^{2j/d}}\right). \end{aligned} \tag{10.6.2}$$

At first glance, this trigonometric-function design looks weird. Before explanations of this design, let us first implement it in the following PositionalEncoding class.

```
#@save
class PositionalEncoding(nn.Module):
    """Positional encoding."""
    def __init__(self, num_hiddens, dropout, max_len=1000):
        super(PositionalEncoding, self).__init__()
        self.dropout = nn.Dropout(dropout)
        # Create a long enough 'P'
        self.P = torch.zeros((1, max_len, num_hiddens))
        X = torch.arange(max_len, dtype=torch.float32).reshape(
            -1, 1) / torch.pow(10000, torch.arange(

```

(continues on next page)

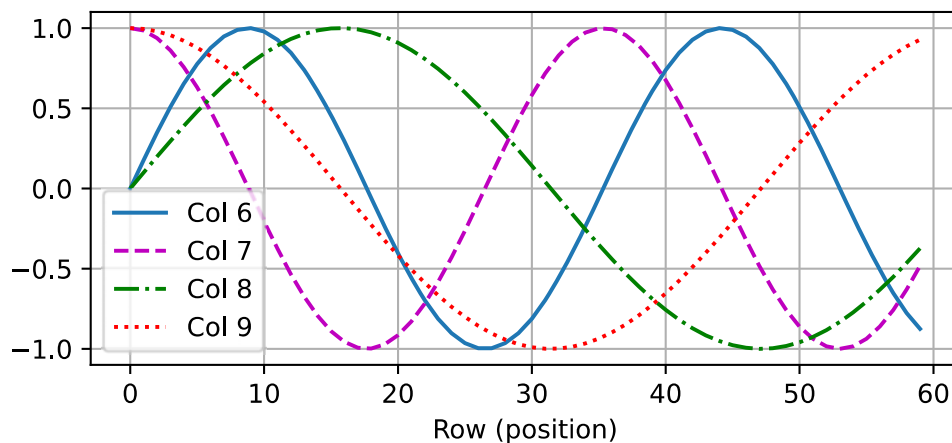
(continued from previous page)

```
0, num_hiddens, 2, dtype=torch.float32) / num_hiddens)
self.P[:, :, 0::2] = torch.sin(X)
self.P[:, :, 1::2] = torch.cos(X)

def forward(self, X):
    X = X + self.P[:, :X.shape[1], :].to(X.device)
    return self.dropout(X)
```

In the positional embedding matrix $\mathbf{P}$, rows correspond to positions within a sequence and columns represent different positional encoding dimensions. In the example below, we can see that the 6th and the 7th columns of the positional embedding matrix have a higher frequency than the 8th and the 9th columns. The offset between the 6th and the 7th (same for the 8th and the 9th) columns is due to the alternation of sine and cosine functions.

```
encoding_dim, num_steps = 32, 60
pos_encoding = PositionalEncoding(encoding_dim, 0)
pos_encoding.eval()
X = pos_encoding(torch.zeros((1, num_steps, encoding_dim)))
P = pos_encoding.P[:, :X.shape[1], :]
d2l.plot(torch.arange(num_steps), P[0, :, 6:10].T, xlabel='Row (position)',
         figsize=(6, 2.5), legend=["Col %d" % d for d in torch.arange(6, 10)])
```



Absolute Positional Information

To see how the monotonically decreased frequency along the encoding dimension relates to absolute positional information, let us print out the binary representations of $0, 1, \dots, 7$. As we can see, the lowest bit, the second-lowest bit, and the third-lowest bit alternate on every number, every two numbers, and every four numbers, respectively.

```
for i in range(8):
    print(f'{i} in binary is {i:>03b}')
```

```
0 in binary is 000
1 in binary is 001
```

(continues on next page)

```

2 in binary is 010
3 in binary is 011
4 in binary is 100
5 in binary is 101
6 in binary is 110
7 in binary is 111

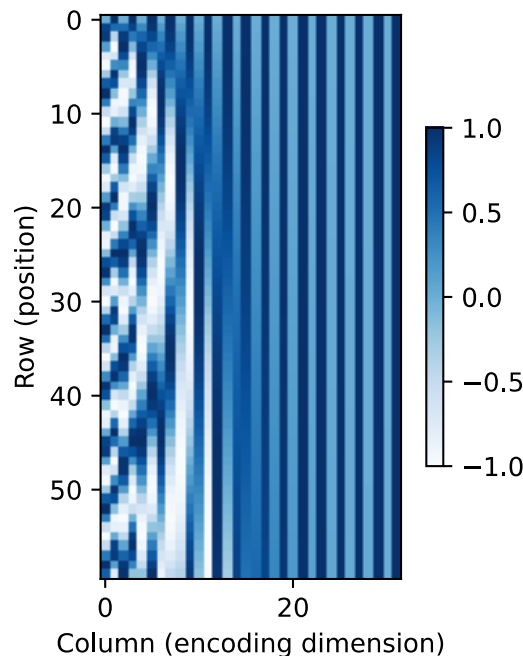
```

In binary representations, a higher bit has a lower frequency than a lower bit. Similarly, as demonstrated in the heat map below, the positional encoding decreases frequencies along the encoding dimension by using trigonometric functions. Since the outputs are float numbers, such continuous representations are more space-efficient than binary representations.

```

P = P[0, :, :].unsqueeze(0).unsqueeze(0)
d2l.show_heatmaps(P, xlabel='Column (encoding dimension)',
                  ylabel='Row (position)', figsize=(3.5, 4), cmap='Blues')

```



Relative Positional Information

Besides capturing absolute positional information, the above positional encoding also allows a model to easily learn to attend by relative positions. This is because for any fixed position offset δ , the positional encoding at position $i + \delta$ can be represented by a linear projection of that at position i .

This projection can be explained mathematically. Denoting $\omega_j = 1/10000^{2j/d}$, any pair of

$(p_{i,2j}, p_{i,2j+1})$ in (10.6.2) can be linearly projected to $(p_{i+\delta,2j}, p_{i+\delta,2j+1})$ for any fixed offset δ :

$$\begin{aligned}
& \begin{bmatrix} \cos(\delta\omega_j) & \sin(\delta\omega_j) \\ -\sin(\delta\omega_j) & \cos(\delta\omega_j) \end{bmatrix} \begin{bmatrix} p_{i,2j} \\ p_{i,2j+1} \end{bmatrix} \\
&= \begin{bmatrix} \cos(\delta\omega_j) \sin(i\omega_j) + \sin(\delta\omega_j) \cos(i\omega_j) \\ -\sin(\delta\omega_j) \sin(i\omega_j) + \cos(\delta\omega_j) \cos(i\omega_j) \end{bmatrix} \\
&= \begin{bmatrix} \sin((i+\delta)\omega_j) \\ \cos((i+\delta)\omega_j) \end{bmatrix} \\
&= \begin{bmatrix} p_{i+\delta,2j} \\ p_{i+\delta,2j+1} \end{bmatrix},
\end{aligned} \tag{10.6.3}$$

where the 2×2 projection matrix does not depend on any position index i .

Summary

- In self-attention, the queries, keys, and values all come from the same place.
- Both CNNs and self-attention enjoy parallel computation and self-attention has the shortest maximum path length. However, the quadratic computational complexity with respect to the sequence length makes self-attention prohibitively slow for very long sequences.
- To use the sequence order information, we can inject absolute or relative positional information by adding positional encoding to the input representations.

Exercises

1. Suppose that we design a deep architecture to represent a sequence by stacking self-attention layers with positional encoding. What could be issues?
2. Can you design a learnable positional encoding method?

10.7 Transformer

We have compared CNNs, RNNs, and self-attention in Section 10.6.2. Notably, self-attention enjoys both parallel computation and the shortest maximum path length. Therefore naturally, it is appealing to design deep architectures by using self-attention. Unlike earlier self-attention models that still rely on RNNs for input representations (Cheng et al., 2016; Lin et al., 2017b; Paulus et al., 2017), the transformer model is solely based on attention mechanisms without any con-volutional or recurrent layer (Vaswani et al., 2017). Though originally proposed for sequence to sequence learning on text data, transformers have been pervasive in a wide range of modern deep learning applications, such as in areas of language, vision, speech, and reinforcement learning.